SAE "Atelier-Projet de développement web 1" / 2026
La billetterie du festival Nancy Rock Vibration

Premier jet

1. Documents de Conception

- Maquettes d'interface : Ecrans principaux (accueil/liste des spectacles, filtres, détail d'une soirée).
- Scénario d'usage : Description textuelle du parcours utilisateur (comment l'utilisateur navigue, filtre et consulte les soirées/spectacles).
- Modèle du domaine métier : Schéma conceptuel / logique représentant les entités du festival (Soirées, Spectacles, Artistes, Lieux, Tarifs).

2. Première Version de l'Application

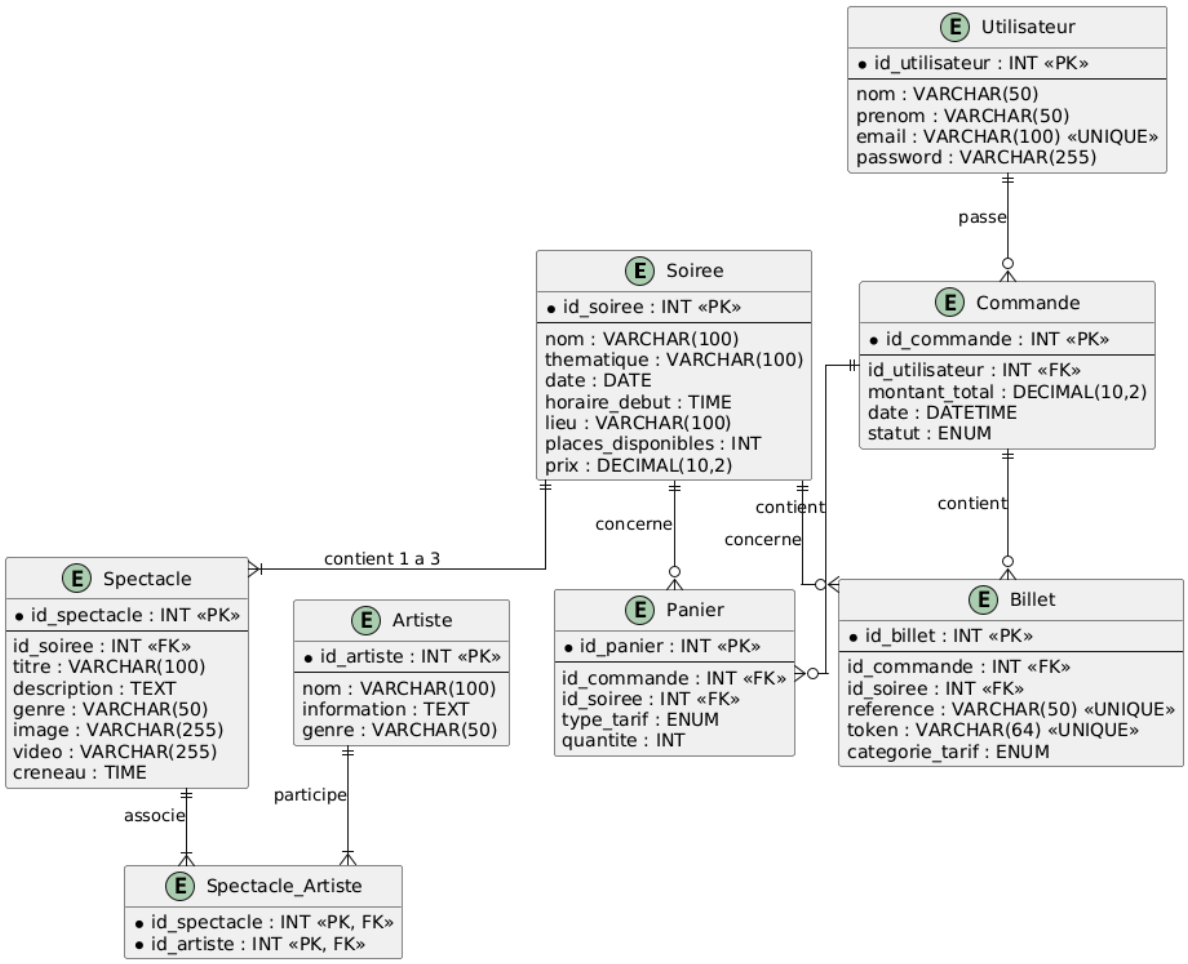
- Base de données & Données :
  - Script SQL de création des tables initiales.
  - Jeu de données de test pour alimenter l'application.
- Backend (API REST PHP / Slim) :
  - Structure de base de l'API avec conteneurisation Docker.
  - Route pour afficher la liste des spectacles (GET /spectacles).
  - Route pour afficher le détail d'une soirée (GET /soirees/{id}).
- Frontend (HTML / CSS / JS) :
  - Interface responsive d'affichage de la liste des spectacles.
  - Intégration des requêtes fetch et des templates JS (ex: Handlebars) pour injecter les données de l'API.
  - Mise en place des filtres de base (par date et par style de musique**).**

Fonctionnalitées à réaliser pour le premier jet

- Fonctionnalité 1 (Affichage liste) : Afficher la liste de tous les spectacles avec pour chacun son titre, sa date, son horaire et une image.
- Fonctionnalité 2 (Affichage détail) : Afficher le détail complet d'une soirée (nom, thématique, date, horaire, lieu, tarifs) ainsi que ses spectacles associés (titre, artistes, description, style de musique, vidéo).
- Fonctionnalité 3 (Navigation) : Pouvoir cliquer sur un spectacle dans la liste générale pour être redirigé vers le détail de sa soirée correspondante.
- Fonctionnalité 4 (Filtre par date) : Filtrer dynamiquement la liste des spectacles selon une date choisie.
- Fonctionnalité 5 (Filtre par style) : Filtrer dynamiquement la liste des spectacles selon leur style de musique (classic rock, blues rock, métal, chanson...).

Domaine métier :

- Soirée : Représente une soirée unique du festival
    - Attributs : id_soiree, nom, thematique, date, horaire début, lieu, places_disponibles, prix
    - Relations :
        - Contient 1 à 3 spectacles
        - Ajout à plusieurs panie
- Spectacle : Représente un spectacle (concert) lors d'une soirée
    - Attributs : id_spectacle, id_soiree, titre, description, genre, images, vidéos, créneau
    - Relations :
        - Un ou plusieurs artistes par spectacle
        - Appartient à une soirée
- Spectacle_Artiste : Représente le fait qu'un spectacle accueil plusieurs artistes
    - Attributs : id_spectacle, id_artiste
    - Relations :
        - Relie une spectacle à plusieurs artistes
- Billet : Représente un billet pour voir un spectacle
    - Attributs : id_billet, id_commande, id_soiree, reference, token, categorie_tarif
    - Relations :
        - Un billet est assigné à une commande
- Utilisateur : Représente le visiteur enregistré
    - Attributs : id_utilisateur, nom, prenom, email, password
    - Relations :
        - Possède un panier
        - Peut commander plusieurs spectacle
        - Peut posséder plusieurs billets
- Commande : Représente une commande en cours ou validé
    - Attributs : id_commande, id_utilisateur, montant_total, date, statut
    - Relations :
        - Une commande peut avoir plusieurs billet
        - Renvoie de billet correspondant à une soirée
- Panier : Représente un panier en cours
    - Attributs : id_panier, id_commande, id_soiree, type_tarif, quantite
    - Relations :
        - Renvoie d'une commande représentant un panier
- Artistes : Représente un groupe de musique
    - Attributs : id_artiste, information, genre
    - Relations :
        - Un groupe peut jouer pour plusieurs spectacle




```
@startuml
hide methods
hide stereotypes
skinparam linetype ortho

entity "Soiree" as soiree {
  * id_soiree : INT <<PK>>
  --
  nom : VARCHAR(100)
  thematique : VARCHAR(100)
  date : DATE
  horaire_debut : TIME
  lieu : VARCHAR(100)
  places_disponibles : INT
  prix : DECIMAL(10,2)
}

entity "Spectacle" as spectacle {
  * id_spectacle : INT <<PK>>
```

```
  --
  id_soiree : INT <<FK>>
  titre : VARCHAR(100)
  description : TEXT
  genre : VARCHAR(50)
  image : VARCHAR(255)
  video : VARCHAR(255)
  creneau : TIME
}

entity "Spectacle_Artiste" as spectacle_artiste {
  * id_spectacle : INT <<PK, FK>>
  * id_artiste : INT <<PK, FK>>
}

entity "Artiste" as artiste {
  * id_artiste : INT <<PK>>
  --
  nom : VARCHAR(100)
  information : TEXT
  genre : VARCHAR(50)
}

entity "Utilisateur" as utilisateur {
  * id_utilisateur : INT <<PK>>
  --
  nom : VARCHAR(50)
  prenom : VARCHAR(50)
  email : VARCHAR(100) <<UNIQUE>>
  password : VARCHAR(255)
}

entity "Commande" as commande {
  * id_commande : INT <<PK>>
  --
  id_utilisateur : INT <<FK>>
  montant_total : DECIMAL(10,2)
  date : DATETIME
  statut : ENUM
}

entity "Panier" as panier {
  * id_panier : INT <<PK>>
  --
  id_commande : INT <<FK>>
  id_soiree : INT <<FK>>
  type_tarif : ENUM
  quantite : INT
```

```
}

entity "Billet" as billet {
  * id_billet : INT <<PK>>
  --
  id_commande : INT <<FK>>
  id_soiree : INT <<FK>>
  reference : VARCHAR(50) <<UNIQUE>>
  token : VARCHAR(64) <<UNIQUE>>
  categorie_tarif : ENUM
}

soiree ||--|{ spectacle : contient 1 a 3
spectacle ||--|{ spectacle_artiste : associe
artiste ||--|{ spectacle_artiste : participe
utilisateur ||--o{ commande : passe
commande ||--o{ panier : contient
soiree ||--o{ panier : concerne
commande ||--o{ billet : contient
soiree ||--o{ billet : concerne

@enduml
```

Scénario d'usage : consulter le programme du festival

SAE Atelier-Projet de développement web 1, billetterie Nancy Rock Vibration (NRV.net)

Ce scénario couvre le premier jet : affichage de la liste des spectacles, filtres par date et par style, consultation du détail d'une soirée (fonctionnalités 1 à 5).

Personnage

Julie, 24 ans, étudiante à Nancy. Elle aime surtout le blues rock et sait qu'elle est libre le samedi de la deuxième semaine du festival. Elle veut savoir ce qui se joue, et où, avant d'acheter un billet. Elle consulte le site depuis son téléphone, dans le tram.

Objectif

Trouver une soirée qui lui plaît, à une date où elle est disponible, et voir toutes les informations utiles (lieu, horaire, tarifs, artistes).

Parcours principal

Étape 1 : arrivée sur le site

Julie ouvre NRV.net sur son téléphone.

- **Ce qu'elle voit :** la page d'accueil avec la liste de tous les spectacles du festival. Chaque spectacle est présenté sous forme de carte : une image, le titre, la date et l'horaire. Sur mobile, les cartes sont affichées les unes sous les autres.
- **Ce que fait l'application :** le front appelle l'API avec `GET /spectacles`, reçoit la liste en JSON et génère les cartes avec un template JS (Handlebars).
- **Fonctionnalité concernée :** 1 (affichage de la liste).

Étape 2 : filtre par style de musique

La liste est longue. Julie choisit "Blues rock" dans le filtre par style, en haut de la page.

- **Ce qu'elle voit :** la liste se met à jour sans rechargement de page. Seuls les spectacles de blues rock restent affichés.
- **Ce que fait l'application :** la liste des styles proposés dans le filtre vient de la base. Le choix de Julie déclenche le filtrage de la liste.
- **Fonctionnalité concernée :** 5 (filtre par style).

Étape 3 : filtre par date

Julie ajoute la date du samedi qui l'intéresse dans le filtre par date.

- **Ce qu'elle voit :** il ne reste que les spectacles de blues rock de ce samedi. Ici, un seul spectacle : "Les Rouleurs du Delta", à 21h.
- **Ce que fait l'application :** les deux filtres se combinent. La liste affichée correspond aux spectacles qui respectent à la fois le style et la date.
- **Fonctionnalité concernée :** 4 (filtre par date).

Étape 4 : accès au détail de la soirée

Julie touche la carte "Les Rouleurs du Delta".

- **Ce qu'elle voit :** la page de détail de la soirée qui contient ce spectacle, et non une page du spectacle seul. En effet, on achète un billet pour une soirée, pas pour un spectacle.
- **Ce que fait l'application :** le spectacle connaît sa soirée (`id_soiree`). Le front appelle `GET /soirees/{id}` et affiche le résultat.
- **Fonctionnalité concernée :** 3 (navigation).

Étape 5 : consultation du détail

Julie fait défiler la page de la soirée.

- **Ce qu'elle voit :**
  - en haut, les informations de la soirée : nom ("Samedi Blues"), thématique, date, horaire de début, lieu et les deux tarifs (normal et réduit) ;
  - en dessous, les 2 ou 3 spectacles de la soirée. Pour chacun : titre, artistes, description, style de musique et vidéo.
- **Ce que fait l'application :** une seule réponse de `GET /soirees/{id}` contient la soirée, son lieu, les tarifs et la liste des spectacles avec leurs artistes et leur style.
- **Fonctionnalité concernée :** 2 (affichage du détail).

Julie découvre aussi qu'un spectacle de classic rock est prévu le même soir. Elle regarde la vidéo et décide que la soirée lui convient. Elle a le tarif réduit en tant qu'étudiante.

Étape 6 : retour à la liste

Julie veut comparer avec d'autres soirées. Elle revient à la liste des spectacles.

- **Ce qu'elle voit :** la liste complète, ou la liste filtrée si les filtres ont été conservés (choix à faire par le groupe).

Cas particuliers

- **Aucun résultat :** si Julie choisit une date où il n'y a pas de blues rock, la page affiche un message clair ("Aucun spectacle ne correspond à vos critères") et un moyen de réinitialiser les filtres, plutôt qu'une page vide.
- **Réinitialiser les filtres :** un bouton permet de revenir à la liste complète en un geste.
- **Soirée introuvable :** si l'identifiant de la soirée n'existe pas, l'API renvoie une erreur 404 et le front affiche un message d'erreur.
- **Ordinateur de bureau :** même parcours, mais les cartes sont affichées en grille sur plusieurs colonnes.

Suite du parcours (hors premier jet)

Depuis la page de la soirée, Julie pourra ajouter des billets à son panier (fonctionnalité 6), puis valider et payer sa commande. Ce sera l'objet d'un scénario séparé.